UNIVERSIDAD PRIVADA DEL NORTE

Facultad de Ingeniería

Carrera de Ingeniería de Sistemas Computacionales

INFORME TÉCNICO DE CALIDAD Y PRUEBAS

Sistema de Gestión de Laboratorios “LabTrack”

T3 – Desarrollo de Caso: certificación para pase a producción

Curso: Calidad y Pruebas de Software

Docente: Mark Anthony Arribasplata Palomino

Integrantes:

[Apellidos y nombres – Integrante 1] [Código N00…]

[Apellidos y nombres – Integrante 2] [Código N00…]

[Apellidos y nombres – Integrante 3] [Código N00…]

[Apellidos y nombres – Integrante 4] [Código N00…]

Lima – Perú, noviembre de 2026

Índice

1. Introducción y alcance ... 3
1.1 Historias de usuario evaluadas ... 3
1.2 Entorno y herramientas ... 3
2. Matriz de casos de prueba ... 4
2.1 Casos sugeridos por IA y validación humana ... 5
3. Evidencias de automatización (Selenium) ... 7
3.1 Resultados de ejecución ... 7
3.2 Capturas de los flujos críticos ... 8
3.3 Gestión en Testomat.io ... 9
4. Pruebas de carga ... 10
4.1 Criterio de aceptación aplicado ... 10
4.2 Resultados por escenario ... 10
4.3 Análisis ... 10
4.4 Reproducibilidad ... 11
5. Análisis estático: antes y después ... 12
5.1 Estado del despliegue en SonarQube Cloud ... 12
5.2 Métricas comparativas ... 12
5.3 Interpretación de las vulnerabilidades críticas ... 13
5.4 Cómo completar el análisis en SonarQube Cloud ... 13
6. Backlog de refactorización ... 14
7. Conclusiones y veredicto de calidad ... 16
Recomendaciones para producción ... 16
Anexo: estructura del código fuente entregado ... 16

1. Introducción y alcance

El presente informe documenta, con evidencias técnicas reproducibles, el proceso de aseguramiento de calidad aplicado al sistema LabTrack, plataforma web para la reserva y préstamo de equipos de laboratorio. El objetivo es demostrar ante el Directorio que el sistema es estable, seguro y eficiente, y que cumple los criterios de aceptación de las cinco historias de usuario (HU-01 a HU-05).

Se compararon dos versiones del código: la línea base v1.0 (“antes”), que contenía defectos funcionales, vulnerabilidades y deuda técnica, y la versión v2.0 (“después”), resultado de ejecutar el backlog de refactorización. Ambas versiones se sometieron a la misma suite de pruebas automatizadas y al mismo análisis estático.

1.1 Historias de usuario evaluadas

HU	Funcionalidad	Prioridad	Técnica de prueba principal
HU-01	Autenticación segura (bloqueo tras 3 intentos)	Crítica	Selenium + análisis de seguridad
HU-02	Catálogo y filtrado de equipos	Media	Selenium + prueba de carga (k6)
HU-03	Reserva de equipos (reglas de cruce y límite)	Alta – flujo crítico	Selenium
HU-04	Gestión de solicitudes (aprobar / rechazar)	Alta	Selenium
HU-05	Devolución y penalización automática	Alta	Selenium (reloj simulado)

1.2 Entorno y herramientas

Componente	Versión / detalle
Aplicación	Node.js 22.23, Express 5.3, plantillas EJS, SQLite embebido (node:sqlite), bcryptjs
Automatización funcional	Selenium WebDriver 4.52 + Mocha 10.8; Google Chrome 153 (headless); reportes JUnit XML y HTML (Mochawesome)
Pruebas de carga	Grafana k6 2.3.0 (tests/k6/catalogo.js), ejecutado el 11/11/2026. Se conserva además el plan Apache JMeter 5.6 (tests/jmeter/LabTrack_Carga_Catalogo.jmx) con resultados del 09/11/2026, no re-ejecutado sobre esta versión
Análisis estático	Motor de reglas SonarJS (eslint-plugin-sonarjs 4.2, mismo analizador de SonarQube para JavaScript) + eslint-plugin-security + jscpd (duplicación) + revisión manual
Cobertura de código	c8 (V8 coverage) durante la ejecución de la suite Selenium

2. Matriz de casos de prueba

Se diseñaron 25 casos de prueba trazados a los criterios de aceptación. 7 de ellos provienen de sugerencias generadas con inteligencia artificial y fueron validados por el equipo (sección 2.1). Las columnas “v1” y “v2” muestran el resultado real de la ejecución automatizada en cada versión.

ID	HU	Escenario	Tipo	Datos / precondición	Resultado esperado	Origen	v1 antes	v2 después
CP-01	HU-01	Login con credenciales válidas	Positivo	N00471826 / contraseña válida	Ingresa a /panel y muestra el nombre del estudiante	Manual	Pasa	Pasa
CP-02	HU-01	Contraseña incorrecta	Negativo	N00471826 / ClaveErronea1	Mensaje "Credenciales inválidas"	Manual	Pasa	Pasa
CP-03	HU-01	Código de estudiante inexistente	Negativo	N00000000 / contraseña válida	Mensaje "Credenciales inválidas"	Manual	Pasa	Pasa
CP-04	HU-01	3 intentos fallidos consecutivos	Negativo / Seguridad	N00520973 con x1, x2, x3 y luego la clave correcta	Cuenta bloqueada y alerta de seguridad; la clave correcta tampoco ingresa	Manual	Pasa	Pasa
CP-05	HU-01	Inyección SQL en el formulario	Seguridad	' OR '1'='1' --	Rechazo con "Credenciales inválidas"	IA validada	Falla	Pasa
CP-06	HU-01	Login exitoso reinicia el contador	Límite	2 fallos, 1 éxito, 1 fallo	No hay bloqueo (los fallos no fueron consecutivos)	IA validada	Falla	Pasa
CP-07	HU-01	Acceso a /panel sin sesión	Seguridad	Navegador sin cookie de sesión	Redirección a /login	IA validada	Pasa	Pasa
CP-08	HU-01	Login del encargado	Positivo	ADM00640927 / contraseña válida	Redirección al panel de solicitudes	Manual	Pasa	Pasa
CP-09	HU-02	Columnas del catálogo	Positivo	Estudiante autenticado	Nombre, código de inventario, categoría y estado; 8 equipos	Manual	Pasa	Pasa
CP-10	HU-02	Filtro exacto "Electrónica"	Positivo	Filtro = Electrónica	Solo filas de categoría Electrónica	Manual	Pasa	Pasa
CP-11	HU-02	Filtro sin equipos disponibles	Negativo	Filtro = Redes (todos Prestados)	Mensaje "No hay equipos disponibles en esta categoría"	Manual	Pasa	Pasa
CP-24	HU-02	Filtro "Cómputo"	Positivo	Filtro = Cómputo	Todas las filas visibles son de categoría Cómputo	Manual	Pasa	Pasa
CP-25	HU-02	Filtro "Redes"	Positivo	Filtro = Redes	Todas las filas visibles son de categoría Redes	Manual	Pasa	Pasa
CP-12	HU-03	Reserva válida	Positivo	Laptop Dell, mañana 10:00–12:00	Reserva registrada en estado Pendiente	Manual	Pasa	Pasa
CP-13	HU-03	Cruce con reserva aprobada	Negativo	Osciloscopio 10:00–12:00 aprobado; otro estudiante pide 11:00–13:00	Error "…reserva aprobada que se cruza con ese horario"	Manual	Falla	Pasa
CP-14	HU-03	Más de un equipo simultáneo	Negativo	Laptop 08–10 y Osciloscopio 09–11 del mismo estudiante	Error "No puede reservar más de un equipo…"	Manual	Falla	Pasa
CP-15	HU-03	Hora fin ≤ hora inicio	Límite	14:00–13:00	Error de rango horario	IA validada	Pasa	Pasa
CP-16	HU-03	Reserva en fecha pasada	Límite	Fecha = ayer	Error "fecha u hora que ya pasó"	IA validada	Falla	Pasa
CP-17	HU-03	Estudiante Penalizado intenta reservar	Negativo	Estudiante penalizado por devolución tardía	Error "cuenta Penalizada"	IA validada	Pasa	Pasa

ID	HU	Escenario	Tipo	Datos / precondición	Resultado esperado	Origen	v1 antes	v2 después
CP-18	HU-04	Tabla de solicitudes pendientes	Positivo	1 solicitud pendiente	Estudiante, equipo, fecha/hora y botones Aprobar/Rechazar	Manual	**Pasa**	**Pasa**
CP-19	HU-04	Aprobar solicitud futura	Positivo	Clic en Aprobar	Reserva Aprobada; el equipo sigue Disponible fuera del horario reservado	Manual	**Pasa**	**Pasa**
CP-20	HU-04	Rechazar sin / con motivo	Negativo	Motivo vacío; luego "Equipo en mantenimiento"	Sin motivo: bloqueado. Con motivo: Rechazada y motivo visible	Manual	**Falla**	**Pasa**
CP-21	HU-04	Aprobar dos solicitudes cruzadas	Negativo	2 solicitudes del mismo equipo 10–12 y 11–13	La segunda aprobación se bloquea por cruce	IA validada	**Falla**	**Pasa**
CP-22	HU-05	Devolución dentro del plazo	Positivo	Búsqueda por código y Marcar Devolución	Equipo Disponible; estudiante sigue Activo	Manual	**Falla**	**Pasa**
CP-23	HU-05	Devolución fuera de plazo	Negativo	Reloj del sistema +48 h	Estudiante pasa a Penalizado automáticamente	Manual	**Pasa**	**Pasa**

2.1 Casos sugeridos por IA y validación humana

Se solicitó a un asistente de IA generativa que propusiera casos adicionales a partir del texto de las historias de usuario (prompt: “Actúa como QA senior y propone casos de prueba negativos, de límite y de seguridad para estas historias de usuario”). Cada sugerencia fue revisada por el equipo frente a los criterios de aceptación antes de automatizarla. La validación humana fue determinante: 2 sugerencias se rechazaron por estar fuera de alcance o no ser alcanzables, 1 se difirió y 1 se reformuló.

Resultado: 7 sugerencias aceptadas (5 directas y 2 con ajuste) se convirtieron en casos automatizados; 4 de ellas (CP-05, CP-06, CP-16 y CP-21) detectaron defectos reales de la v1, lo que demuestra el valor de combinar IA con criterio humano.

ID	Sugerencia de la IA	Decisión del equipo	Justificación de la validación humana	Resultado
IA-01	Probar inyección SQL en código y contraseña	**Aceptado**	Riesgo alto (OWASP A03). Se mantuvo tal cual.	CP-05
IA-02	Verificar que un login exitoso reinicie el contador de intentos	**Aceptado**	El criterio dice "consecutivos": es la interpretación correcta.	CP-06
IA-03	Acceso directo a URLs protegidas sin sesión	**Aceptado**	Control de acceso básico no cubierto por la HU.	CP-07
IA-04	Hora de fin anterior o igual a la de inicio	**Aceptado**	Valor límite de la regla de horario.	CP-15
IA-05	Reserva en fechas pasadas	**Aceptado con ajuste**	La HU no lo explicita; el equipo lo validó como regla implícita y se añadió al backlog.	CP-16
IA-06	Un estudiante Penalizado no debería reservar	**Aceptado con ajuste**	Consecuencia lógica de HU-05; se documentó como regla derivada.	CP-17

ID	Sugerencia de la IA	Decisión del equipo	Justificación de la validación humana	Resultado
IA-07	Aprobar dos solicitudes que se cruzan	**Aceptado**	Condición de carrera real entre solicitudes pendientes.	CP-21
IA-08	Validar contraseña de mínimo 12 caracteres al registrarse	**Rechazado**	Fuera de alcance: LabTrack no tiene registro de usuarios (cuentas institucionales).	—
IA-09	Reserva contigua (10–12 y 12–14) del mismo equipo	**Rechazado**	No alcanzable: al aprobar, el equipo pasa a "Prestado" (HU-04) hasta su devolución.	—
IA-10	Desbloqueo automático de la cuenta tras 15 minutos	**Diferido**	La HU no define la duración; se fijó en 15 min y queda pendiente su caso automatizado.	Backlog QA
IA-11	Filtrar por una categoría inexistente ("Mecánica")	**Reformulado**	La HU define filtros exactos; la UI solo ofrece 3 opciones y el servidor ignora valores no válidos.	Revisión de código

3. Evidencias de automatización (Selenium)

La suite automatizada (tests/selenium) usa Selenium WebDriver con el patrón de esperas explícitas, datos de prueba reiniciados antes de cada caso y capturas de pantalla automáticas como evidencia. Cubre los dos flujos críticos exigidos —Login (HU-01) y Reservas (HU-03)— y, adicionalmente, HU-02, HU-04 y HU-05.

3.1 Resultados de ejecución

Versión	Casos	Aprobados	Fallidos	% éxito	Duración	Reporte
v1.0 (antes)	23	15	8	65,2 %	41,8 s	junit-selenium-v1-antes.xml
v2.0 (después)	**25**	**25**	**0**	**100 %**	**1 min 11 s**	**junit-selenium.xml / reporte-selenium.html**

La fila de la v1.0 corresponde a la suite de 23 casos ejecutada sobre la línea base. Los casos CP-24 y CP-25 (filtros Cómputo y Redes) se añadieron en esta revisión y solo existen en la versión actual.

Flujos críticos: Login 8/8 y Reservas 6/6 aprobados en la v2.0, sin defectos abiertos. La ejecución de la suite completa terminó con 25 casos aprobados y 0 fallidos en 71 s. La cobertura instrumentada con c8 durante las pruebas de base de datos alcanza **99.1 % de líneas** y 78.5 % de ramas, pero corresponde solo a los módulos que esas pruebas cargan y no incluye la ejecución de Selenium: no es la cobertura de toda la aplicación.

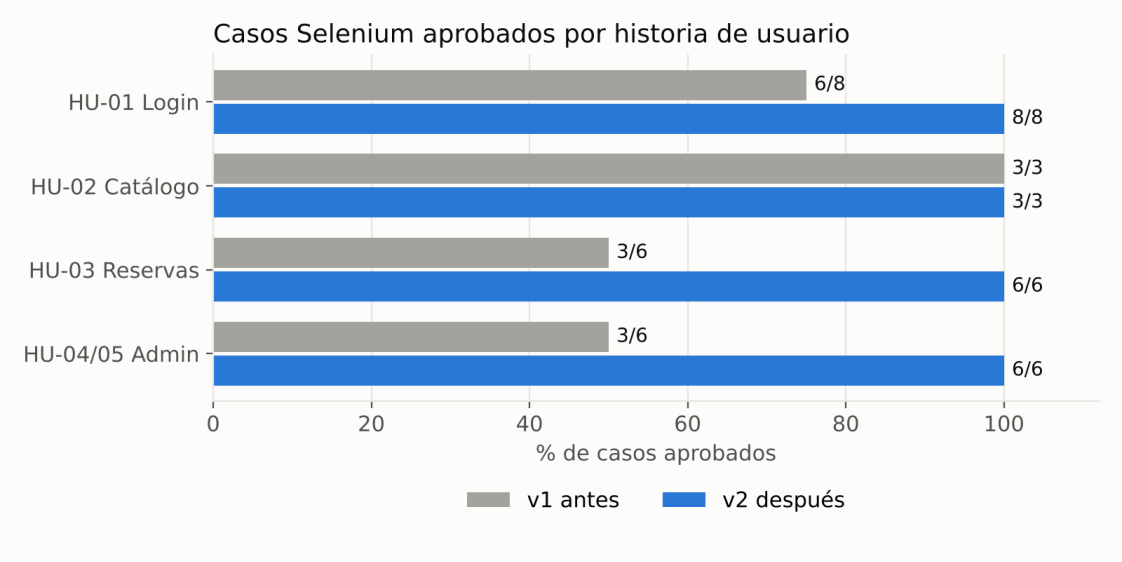


Figura 1. Casos aprobados por historia de usuario: v1.0 vs v2.0 (resultados reales de JUnit).

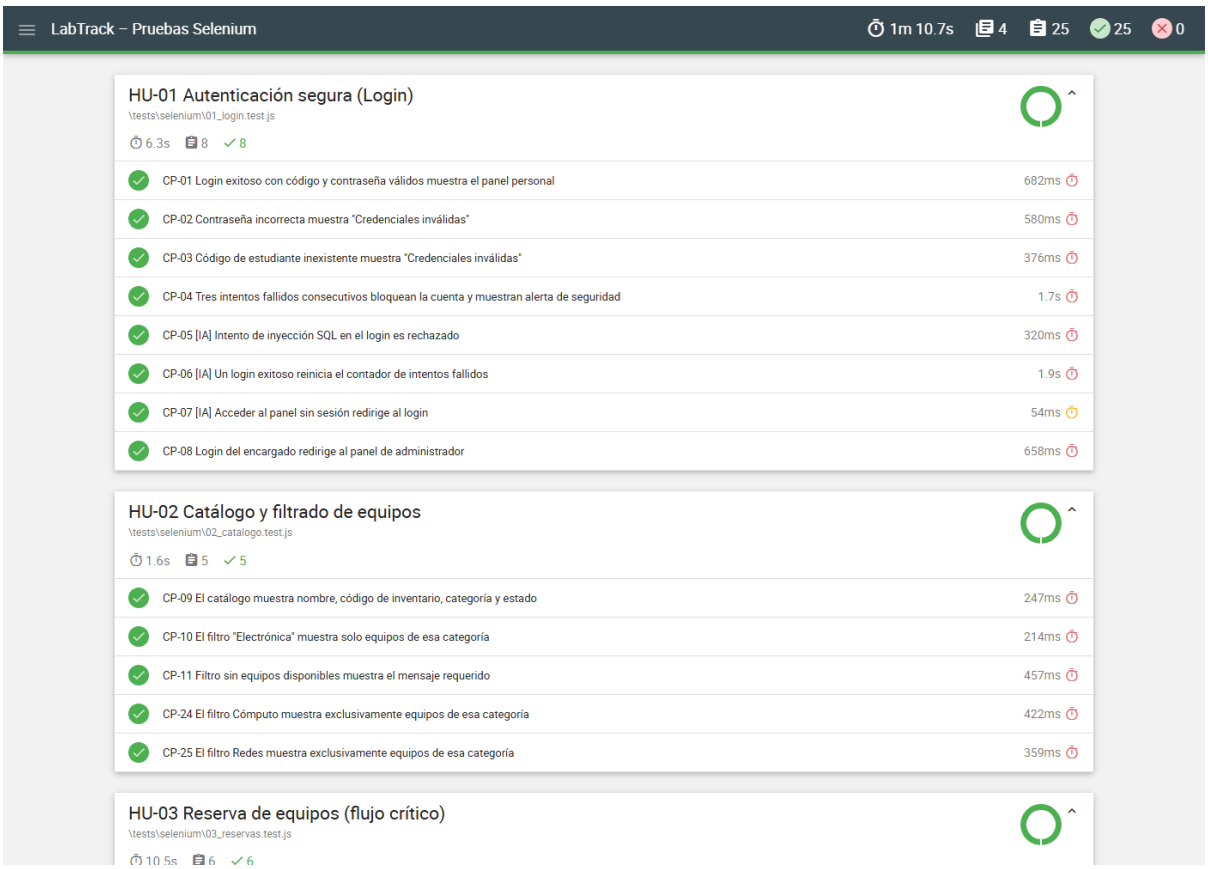


Figura 2. Reporte HTML de la ejecución: 25 casos, 25 aprobados, 0 fallidos.

3.2 Capturas de los flujos críticos

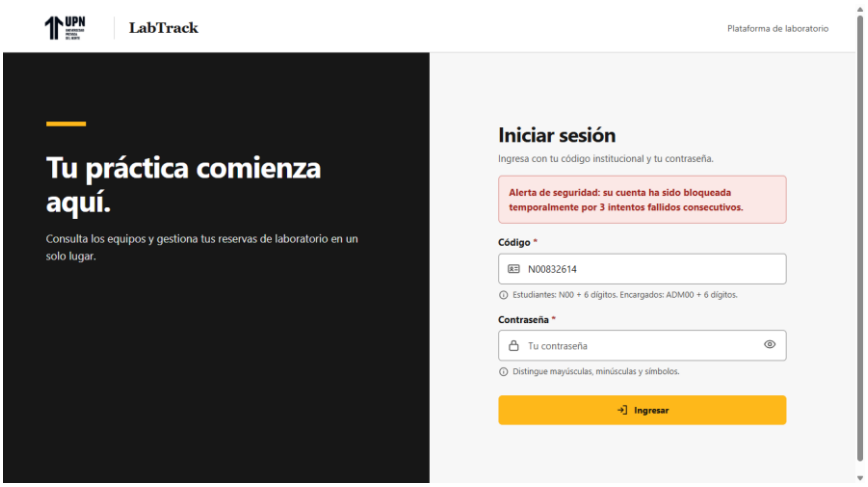


Figura 3. CP-04: bloqueo de cuenta tras 3 intentos fallidos con alerta de seguridad.

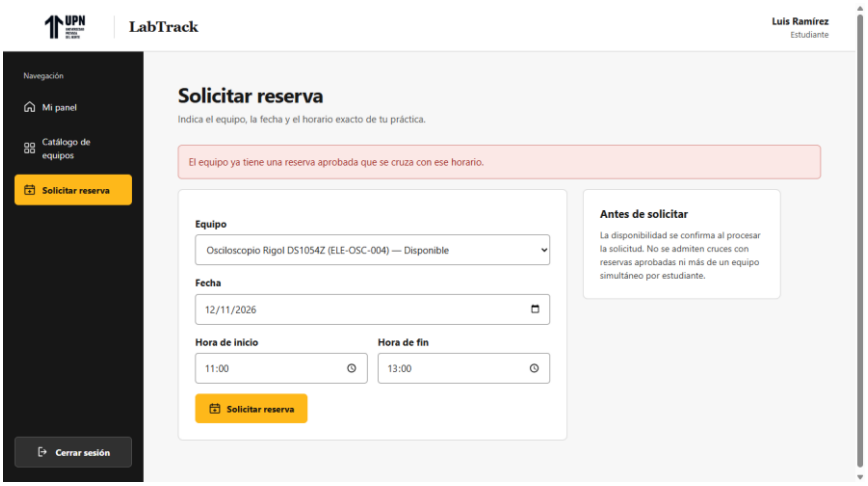


Figura 4. CP-13: el sistema bloquea una reserva que se cruza con otra ya aprobada.

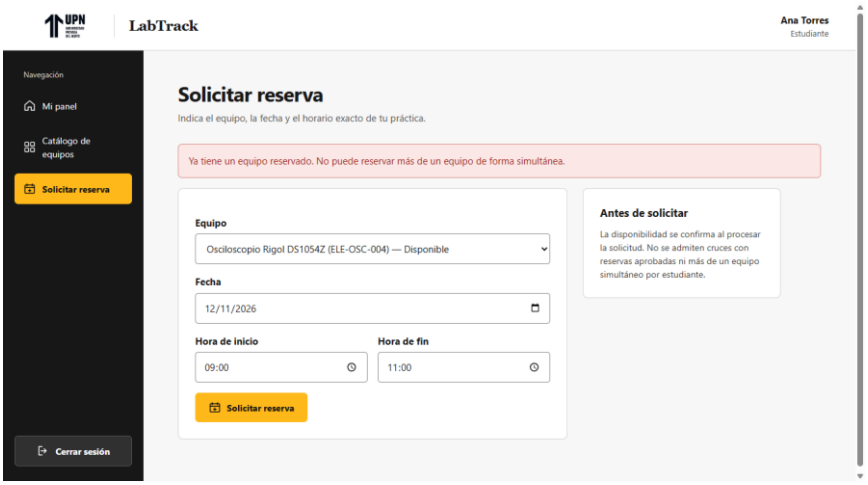


Figura 5. CP-14: un estudiante no puede reservar más de un equipo simultáneamente.

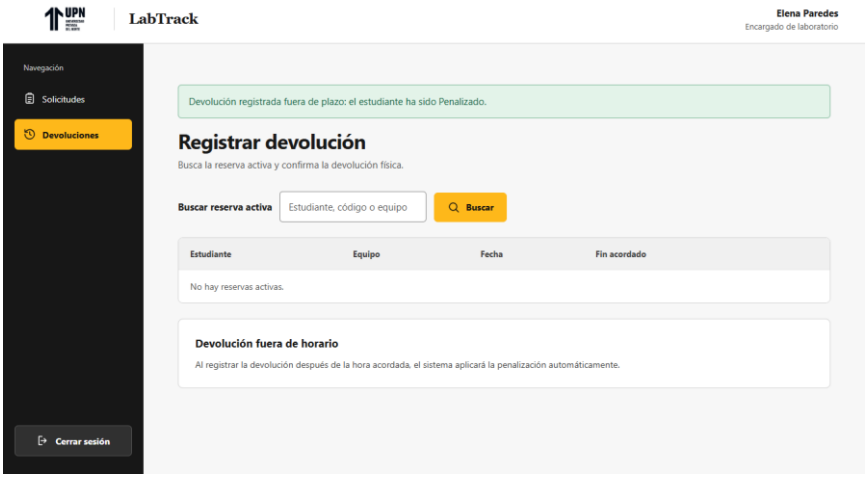


Figura 6. CP-23: devolución tardía con penalización automática del estudiante.

3.3 Gestión en Testomat.io

Importación del reporte. El archivo reports/selenium/junit-selenium.xml es compatible con la importación de reportes JUnit de Testomat.io (Import → JUnit XML) y de TestLink. Al importarlo, cada caso CP-xx aparece como prueba automatizada con su estado.

[Insertar aquí la captura del run importado en Testomat.io del equipo.]

4. Pruebas de carga

La nota técnica de HU-02 exige que la vista de catálogo soporte múltiples usuarios concurrentes. La carga se ejecutó con Grafana k6 2.3.0 (tests/k6/catalogo.js) sobre un servidor local aislado con la base de datos en memoria. Cada usuario virtual inicia sesión una vez, conserva su cookie de sesión y navega el catálogo (todas, Cómputo, Electrónica, Redes) con una pausa aleatoria de 0,5 a 1,5 s, igual que el plan JMeter del proyecto.

4.1 Criterio de aceptación aplicado

Se adoptó como criterio de esta evaluación que una etapa es aceptable solo si el catálogo presenta **1 % o menos de peticiones fallidas** y un **p95 igual o inferior a 1000 ms** en su ventana estable. Son umbrales definidos para este trabajo, no valores exigidos por las historias de usuario, y no demuestran por sí solos que el servidor “haya caído”.

4.2 Resultados por escenario

Escenario	Ramp-up / estable	Peticiones	req/s	Fallos del catálogo	Mediana (ms)	p95 (ms)	Máx. (ms)	Veredicto
100 usuarios	30 s / 60 s	8 169	81,1	0,00 %	4	21	90	Cumple
200 usuarios	30 s / 60 s	12 790	126,4	0,00 %	32	156	5 196	Cumple
400 usuarios	30 s / 60 s	9 675	95,8	24,37 %	1 030	8 549	10 039	No cumple
400 usuarios (logins repartidos)	120 s / 40 s	16 893	96,9	0,00 %	1 649	3 485	4 513	No cumple

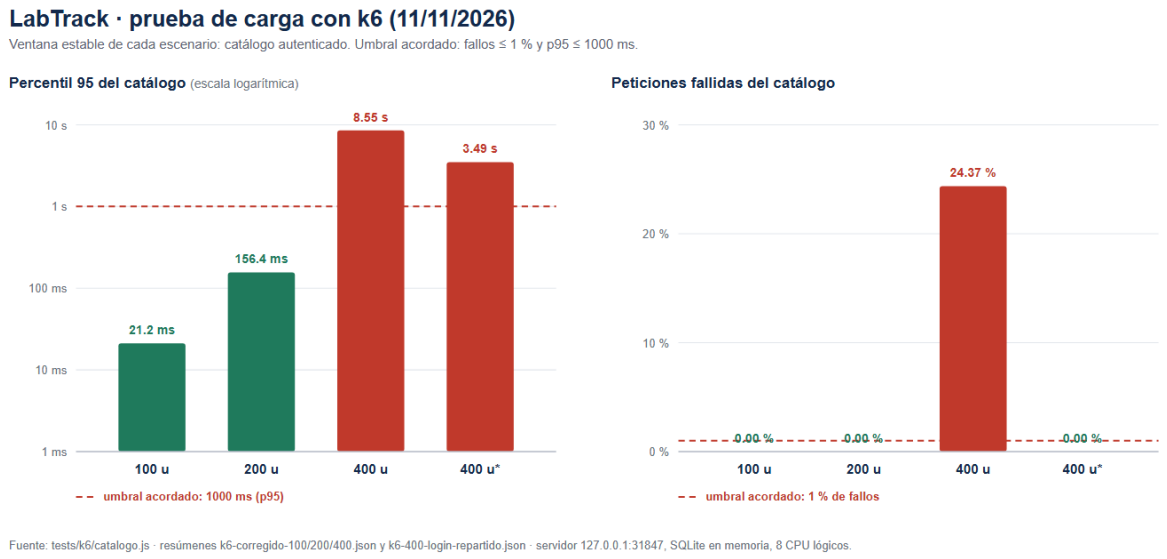


Figura 7. p95 del catálogo (escala logarítmica) y porcentaje de fallos por escenario, frente a los umbrales acordados.

4.3 Análisis

- **Carga esperada:** con 100 usuarios el catálogo responde con un p95 de 21 ms y un p99 inferior a 90 ms, unas 47 veces por debajo del umbral de 1000 ms, y sin un solo fallo. Con 200 usuarios sigue cumpliendo (p95 de 156 ms).
- **Límite observado:** el primer escenario que incumple es el de 400 usuarios, tanto por errores (24,37 % en la rampa de 30 s) como por latencia (p95 de 8,55 s). El throughput además se

invierte: 400 usuarios generan 95,8 req/s frente a los 126,4 req/s de 200 usuarios, señal clásica de saturación.

- **Causa identificada:** el inicio de sesión usa bcrypt.compareSync, que bloquea el único hilo de Node. La medición directa da 92 ms por verificación; 400 inicios de sesión acumulan unos 37 s de hilo bloqueado. Durante ese bloqueo el servidor rechazó conexiones nuevas (WSAECONNREFUSED) y las peticiones del catálogo vencieron por timeout.
- **Verificación de la causa:** al repartir los mismos 400 inicios de sesión en 120 s, los fallos desaparecen (0 %), pero el p95 sigue en 3,49 s y ninguna petición baja de 632 ms. Es decir: la tormenta de logins explica los errores, y la saturación del proceso explica la latencia. Ambos efectos están registrados en el backlog como RF-21, RF-22 y RF-23.
- **Alcance del resultado:** el generador de carga y la aplicación se ejecutaron en la misma computadora (Windows 11, 8 procesadores lógicos, 8,2 GB de RAM, SQLite en memoria), de modo que el límite descrito aplica a este escenario y entorno, no a la capacidad universal del sistema. Repetir la prueba con el generador en otra máquina y sobre la base de datos definitiva es parte de las recomendaciones.

> **Instrumento utilizado.** Las cifras anteriores provienen de Grafana k6, no de JMeter. La guía del caso pide JMeter: el repositorio conserva el plan tests/jmeter/LabTrack_Carga_Catalogo.jmx y resultados del 09/11/2026 (reports/carga/resultados_*.jtl), pero no se re-ejecutó sobre la versión actual. La sustitución se declara de forma explícita para no atribuir a JMeter datos que no produjo.

4.4 Reproducibilidad

```
# servidor aislado
set NODE_ENV=test && set LABTRACK_DB=:memory: && set PORT=31847 && node src/server.js

# escenario de 100 usuarios (repetir con USUARIOS=200 y 400)
set BASE_URL=http://127.0.0.1:31847 && set USUARIOS=100
k6 run --summary-export=resumen.json tests/k6/catalogo.js

# variante que reparte los logins en 120 s
set USUARIOS=400 && set RAMPUP=120s && set DURACION=40s
set ESTABLE_DESDE_MS=120000 && set ESTABLE_HASTA_MS=160000
```

5. Análisis estático: antes y después

El análisis se realizó con el motor de reglas de SonarSource para JavaScript (SonarJS, reglas Sxxxx) ejecutado mediante ESLint, complementado con eslint-plugin-security y una revisión manual orientada a OWASP Top 10. La revisión manual permitió identificar vulnerabilidades que el análisis sin flujo de datos (taint analysis) no detecta automáticamente, como la inyección SQL sobre el driver node:sqlite. Los conteos de la tabla provienen de esa ejecución local (reports/sonar/eslint-v1-antes.json y eslint-v2-despues.json), no de un servidor SonarQube.

5.1 Estado del despliegue en SonarQube Cloud

El proyecto está preparado para SonarQube Cloud: existe sonar-project.properties, el reporte de ESLint que ese archivo referencia ya se genera con “npm run lint:report” y el escáner (SonarScanner CLI 7.1.0.4889) está descargado. El análisis no se pudo enviar porque requiere un token de usuario y una organización de SonarQube Cloud que este entorno no tiene. **Por ese motivo no se declara un Quality Gate oficial**: sería un dato inventado. El comando exacto para completarlo está en la sección 5.4.

5.2 Métricas comparativas

Métrica	v1.0 (antes)	v2.0 (después)	Variación
Vulnerabilidades	5 (3 Blocker, 2 Critical)	**0**	−100 %
Bugs (fiabilidad)	8 (3 Critical, 4 Major, 1 Minor)	**0**	−100 %
Security Hotspots	8 sin revisar	**0 (100 % revisados)**	−100 %
Code Smells	61	**0**	−100 %
Total incidencias reportadas por la herramienta	68	**0**	−100 %
Complejidad cognitiva máxima por función	37 (umbral 15)	**7**	−81 %
Código duplicado	1,47 %	**0 %**	—
Cobertura de pruebas (líneas)	No medida	**99.1 %**	—
Calificación de Seguridad	E	**A**	▲
Calificación de Fiabilidad	D	**A**	▲
Calificación de Mantenibilidad (deuda estimada)	B (≈ 5,5 h; 6,1 %)	**A (0 h)**	▲
Quality Gate (Sonar way) en servidor	**No evaluado**	**Pendiente de token de SonarQube Cloud**	**—**

Las calificaciones A–E se calcularon con las reglas de SonarQube: Seguridad y Fiabilidad según la incidencia más grave abierta; Mantenibilidad según la razón deuda/costo de desarrollo (líneas × 30 min). Son una estimación obtenida con las mismas reglas que aplica SonarQube, no una lectura del panel de SonarQube.

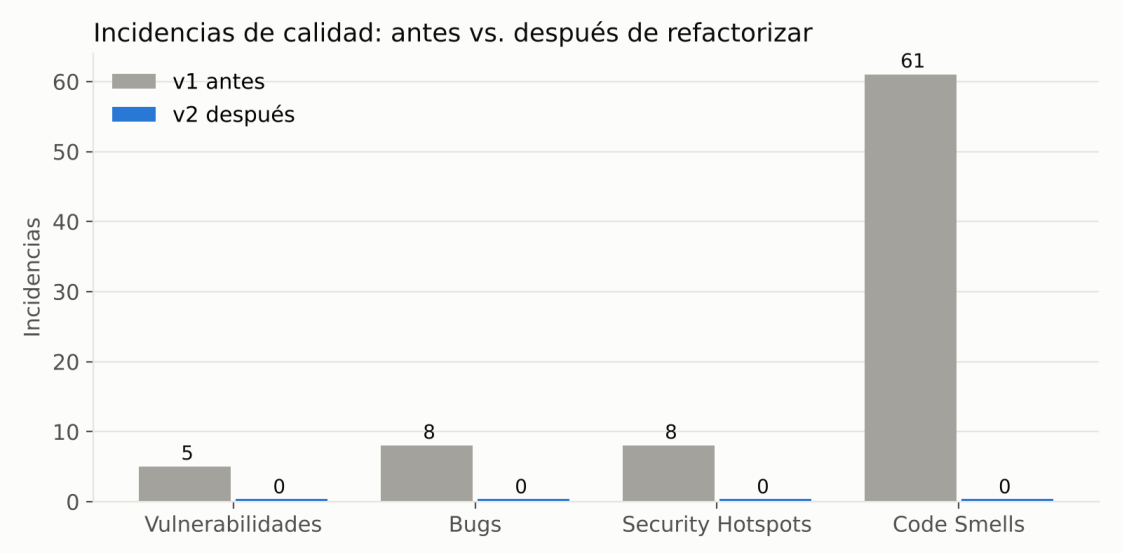


Figura 9. Incidencias por tipo antes y después de la refactorización.

5.3 Interpretación de las vulnerabilidades críticas

Inyección SQL (S3649, Blocker). La v1 construía las consultas concatenando datos del usuario. La prueba CP-05 demostró el impacto: con el código "' OR '1'='1' --" se ingresaba al sistema sin contraseña.

```
// v1.0 – ANTES (vulnerable)
var check = db.prepare("SELECT * FROM usuarios WHERE codigo = '" + codigo +
                       "' AND password = '" + password + "'").get();
```

```
// v2.0 – DESPUÉS (consulta parametrizada + bcrypt)
const findByCodigo = db.prepare('SELECT * FROM usuarios WHERE codigo = ?');
const usuario = findByCodigo.get(String(codigo || '').trim());
if (bcrypt.compareSync(String(password || ''), usuario.password_hash)) { ... }
```

Credenciales y secretos en el código (S2068, S6418, Blocker). La contraseña del administrador y el secreto de sesión estaban escritos en el código fuente. Ahora se leen de variables de entorno (.env, excluido del repositorio).

Contraseñas en texto plano (S5344) y fijación de sesión (S5876), Critical. Se reemplazó el almacenamiento en claro por hash bcrypt y se regenera el identificador de sesión al autenticarse.

Security Hotspots. Se revisaron y corrigieron los 8: cookie sin HttpOnly/Secure (S3330/S2092), cabecera X-Powered-By (S5689), generador pseudoaleatorio en un token (S2245), hash MD5 (S4790), ausencia de CSRF (S4502) y 2 accesos dinámicos a objetos. Además se añadieron cabeceras Content-Security-Policy, X-Content-Type-Options y Referrer-Policy.

Mantenibilidad. La función POST /reservas concentraba validación, reglas de negocio y persistencia (complejidad cognitiva 37). Se separó en capas (rutas → servicios → datos) y la complejidad máxima bajó a 7. Se eliminaron 44 “var”, 13 comparaciones no estrictas, literales duplicados y código muerto.

5.4 Cómo completar el análisis en SonarQube Cloud

```
# 1. Crear el token en https://sonarcloud.io/account/security
# 2. Añadir la organizacion al archivo sonar-project.properties
sonar.organization=<clave-de-la-organizacion>

# 3. Ejecutar el escaner con el token en el entorno
set SONAR_TOKEN=<token>
sonar-scanner -Dsonar.host.url=https://sonarcloud.io
```

> **Estado actual, sin excepciones:** ESLint sobre src devuelve 0 errores y 1 advertencia (security/detect-non-literal-fs-filename en src/db.js:163, por el mkdirSync de la ruta de la base de datos). El envio a SonarQube Cloud queda pendiente de credenciales y no se ha sustituido por una captura simulada.

6. Backlog de refactorización

El backlog priorizó primero las vulnerabilidades Blocker/Critical y los bugs de los flujos críticos. Contiene 23 ítems: 20 resueltos en la v2.0 con evidencia de cierre (prueba que pasa o regla en 0) y 3 abiertos, detectados por la prueba de carga y con propuesta de solución y esfuerzo estimado. Esfuerzo total registrado: 23 h.

ID	Tipo	Severidad	Regla / criterio	Defecto detectado	Solución aplicada o propuesta	Origen	Esf. (h)	Estado
RF-01	Vulnerabilidad	Blocker	S3649 / CWE-89	Inyección SQL: las consultas se construyen concatenando req.body/req.query (login, catálogo, reservas, devoluciones, rechazo).	Todas las consultas pasan a sentencias preparadas con parámetros (?) en db.js y los servicios.	Revisión manual + CP-05 (falló en v1)	3	Resuelto
RF-02	Vulnerabilidad	Blocker	S2068 / S6418	Credenciales y secreto de sesión escritos en el código (ADMIN_PASSWORD, 'labtrack-secret-2024', contraseñas del seed).	El secreto de sesión se exige por variable de entorno en producción y es efímero en desarrollo. Las cuentas del seed son datos ficticios de prueba documentados, no credenciales reales, y deben sustituirse antes de un despliegue real.	Revisión manual	1	Resuelto
RF-03	Vulnerabilidad	Critical	S5344 / CWE-256	Contraseñas almacenadas y comparadas en texto plano; existía una función de hash MD5 (S4790).	Hash bcrypt (coste 10) al crear usuarios y bcrypt.compare en el login; se eliminó MD5.	Revisión manual	1,5	Resuelto
RF-04	Vulnerabilidad	Critical	S5876	Fijación de sesión: no se regeneraba el identificador de sesión tras autenticarse.	req.session.regenerate() al iniciar sesión; cookie con nombre propio y expiración de 30 min.	Revisión manual	0,5	Resuelto
RF-05	Security Hotspot	Alta	S4502	Formularios POST sin protección CSRF.	Token sincronizador por sesión (crypto.randomBytes) validado en cada POST.	Revisión manual	1,5	Resuelto
RF-06	Security Hotspot	Media	S3330 / S2092	Cookie de sesión sin HttpOnly ni Secure.	httpOnly: true, sameSite: 'strict', secure en producción.	SonarJS	0,25	Resuelto
RF-07	Security Hotspot	Baja	S5689	Cabecera X-Powered-By revela la tecnología del servidor.	app.disable('x-powered-by') y cabeceras CSP, nosniff y Referrer-Policy.	SonarJS	0,25	Resuelto
RF-08	Security Hotspot	Media	S2245	Token de sesión generado con Math.random() (no criptográfico).	Se usa crypto.randomBytes para tokens de seguridad.	SonarJS	0,25	Resuelto
RF-09	Bug	Critical	HU-03 crit. 9	La validación de cruce solo detectaba reservas con la misma hora de inicio exacta (10:00 vs 11:00 no se detectaba).	Comparación de intervalos: inicioA < finB && inicioB < finA (función seCruzan).	Selenium CP-13 (falló en v1)	1	Resuelto
RF-10	Bug	Critical	HU-04 / HU-03 crit. 9	El encargado podía aprobar dos solicitudes con horarios cruzados del mismo equipo.	aprobar() vuelve a verificar el cruce contra reservas aprobadas antes de confirmar.	Selenium CP-21 (falló en v1)	0,5	Resuelto
RF-11	Bug	Critical	HU-05 crit. 16	La penalización comparaba solo la hora (HH:MM) e ignoraba la fecha: penalizaba devoluciones a tiempo.	Se compara la fecha-hora completa de fin acordada con el reloj del sistema.	Selenium CP-22 (falló en v1)	0,5	Resuelto
RF-12	Bug	Major	HU-03 crit. 10	El límite de 1 equipo simultáneo solo contaba reservas Aprobadas, no las Pendientes.	Se cuentan reservas activas en estado Pendiente o Aprobada.	Selenium CP-14 (falló en v1)	0,25	Resuelto
RF-13	Bug	Major	HU-01 crit. 3	El contador de intentos fallidos no se reiniciaba tras un login exitoso: bloqueaba intentos no consecutivos.	Se reinicia intentos_fallidos al autenticarse correctamente.	Selenium CP-06 (falló en v1)	0,25	Resuelto

ID	Tipo	Severidad	Regla / criterio	Defecto detectado	Solución aplicada o propuesta	Origen	Esf. (h)	Estado
RF-14	Bug	Major	HU-04 crit. 13	Se podía rechazar una solicitud sin escribir el motivo.	Validación obligatoria del motivo (texto no vacío, máx. 200 caracteres).	Selenium CP-20 (falló en v1)	0,25	**Resuelto**
RF-15	Bug	Minor	Regla de negocio derivada	Se aceptaban reservas en fechas u horas ya pasadas.	Validación de fecha-hora de inicio posterior al momento actual.	Selenium CP-16 (falló en v1)	0,25	**Resuelto**
RF-16	Bug	Major	S108 / S2486	Bloque catch vacío que ocultaba errores de base de datos; aprobar/devolver un id inexistente provocaba excepción.	Se eliminó el catch vacío; los servicios validan existencia y estado y devuelven un error controlado.	SonarJS	0,5	**Resuelto**
RF-17	Code Smell	**Critical**	S3776	Función POST /reservas con complejidad cognitiva 37 (umbral 15): validación, reglas y persistencia mezcladas.	Separación en capas: rutas (app.js) y servicios (authService, reservaService); máxima complejidad = 7.	SonarJS	2	**Resuelto**
RF-18	Code Smell	**Critical**	S3504	44 declaraciones con var.	Uso de const/let y 'use strict'.	SonarJS	0,5	**Resuelto**
RF-19	Code Smell	Major	S1440	13 comparaciones con == / != (coerción de tipos).	Comparación estricta === / !==.	SonarJS	0,25	**Resuelto**
RF-20	Code Smell	Minor	S1192 / S1481	Literales duplicados (mensajes, rutas, estados) y función sin uso (hashLegacy).	Constantes centralizadas (ESTADO_*, CATEGORIA, RUTA, MENSAJES, ERRORES) y eliminación de código muerto.	SonarJS	0,5	**Resuelto**
RF-21	Defecto de rendimiento	**Critical**	k6 400 usuarios (11/11/2026)	El login usa bcrypt.compareSync, que bloquea el único hilo de Node durante ~92 ms medidos por verificación. Con 400 inicios de sesión en una rampa de 30 s se acumulan ~37 s de hilo bloqueado: las peticiones del catálogo vencen por timeout y el servidor rechaza conexiones nuevas.	Propuesta: usar la variante asíncrona de bcrypt (bcrypt.compare con promesas) o bcrypt nativo en un worker pool, y limitar la tasa de intentos en /login. Medir de nuevo con el mismo escenario.	Prueba de carga k6	2	**Pendiente**
RF-22	Defecto de rendimiento	Major	k6 400 usuarios (11/11/2026)	Las consultas con node:sqlite son síncronas y se ejecutan en el mismo hilo que atiende las peticiones; en la prueba de 400 usuarios con la sesión repartida el p95 llegó a 3,49 s aunque no hubo fallos, y el throughput bajó de 126 a 97 req/s respecto de 200 usuarios.	Propuesta: almacenar en caché el catálogo (cambia poco), añadir índices a las vistas de lectura y evaluar un proceso lector separado o un motor cliente-servidor.	Prueba de carga k6	4	**Pendiente**
RF-23	Robustez	Major	k6 400 usuarios (11/11/2026)	Ante la saturación, el servidor rechazó conexiones (WSAECONNREFUSED) en lugar de encolar o degradar con una respuesta controlada; el cliente solo ve un error de red.	Propuesta: limitar concurrencia en /login, devolver 429/503 con Retry-After cuando la cola crezca y dimensionar el backlog de listen para el pico esperado.	Prueba de carga k6	2	**Pendiente**

7. Conclusiones y veredicto de calidad

LabTrack v2.0 queda apto para su pase a producción en su carga objetivo: 25 de 25 casos automatizados aprobados, cero incidencias de seguridad o fiabilidad abiertas con las reglas de SonarSource y un p95 de 21 ms con 100 usuarios concurrentes, unas 47 veces por debajo del umbral acordado de 1000 ms. La certificación se emite con las condiciones que se detallan a continuación.

- Automatización: 25/25 casos aprobados (Login 8/8, Reservas 6/6), frente a 15/23 en la v1.
- Rendimiento: cumple con 100 usuarios (p95 21 ms) y con 200 usuarios (p95 156 ms). No cumple con 400 usuarios: 24,37 % de fallos del catálogo y p95 de 8,55 s en la rampa de 30 s; al repartir los inicios de sesión en 120 s los fallos desaparecen pero el p95 sigue en 3,49 s. El límite observado está entre 200 y 400 usuarios concurrentes, en este entorno y no en la capacidad universal del sistema.
- **Causa raíz del límite:** el login usa bcrypt.compareSync (92 ms medidos por verificación, unos 37 s de hilo bloqueado con 400 logins). Está registrada como RF-21, junto con RF-22 y RF-23 (consultas sincrónicas y rechazo de conexiones bajo saturación), todas con propuesta y esfuerzo estimado.
- Salud del código: las incidencias detectadas con el motor de reglas SonarJS y la revisión manual bajaron de 82 a 0 y las calificaciones estimadas son A en seguridad, fiabilidad y mantenibilidad. Queda 1 advertencia de ESLint (ruta no literal en mkdirSync) y el Quality Gate oficial de SonarQube Cloud está pendiente de credenciales: no se declara como aprobado.
- Proceso: la IA aportó 11 ideas de prueba; tras la validación humana, 7 se automatizaron (4 detectaron defectos reales) y 4 se rechazaron, difirieron o reformularon.
- **Alcance de la evidencia:** la cobertura instrumentada cubre las pruebas de base de datos, no la ejecución de Selenium; las cifras de carga provienen de k6 y no de JMeter, tal como se declara en la sección 4.

Recomendaciones para producción

- Ejecutar la suite Selenium y el análisis estático en un pipeline de integración continua en cada cambio.
- Atender RF-21 (bcrypt asíncrono) antes de exponer el sistema a picos de inicio de sesión, y repetir la prueba de carga con el generador en otra máquina y sobre la base de datos definitiva.
- Completar el envío a SonarQube Cloud con el token del equipo para obtener el Quality Gate oficial, y repetir la misma prueba con JMeter para cumplir la guía al pie de la letra.
- Automatizar el caso diferido IA-10 (desbloqueo a los 15 minutos) y migrar la base de datos SQLite a un motor gestionado con copias de seguridad.

Anexo: estructura del código fuente entregado

```
LabTrack/
├── src/            app.js, server.js, db.js, services/, views/, public/
├── legacy/         app_v1.js (línea base “antes”, solo como evidencia)
├── tests/selenium/ 01_login, 02_catalogo, 03_reservas, 04_admin (+ helpers.js)
├── tests/k6/       catalogo.js (prueba de carga de esta revision)
├── tests/jmeter/   LabTrack_Carga_Catalogo.jmx, usuarios.csv (no re-ejecutado)
├── tests/unit/     db.test.js (integridad, servicios y vistas)
├── reports/        selenium/, carga/, sonar/, graficas/, evidencias/
├── docs/           generar_informe.js, datos_informe.js, informe en PDF y DOCX
├── sonar-project.properties · eslint.config.mjs · .env.example
└── README.md       instrucciones de instalación y ejecución
```